TRIBHUVAN UNIVERSITY

Institute of Engineering, Purwanchal Campus

Department of Electronics and Computer Engineering

Dharan, Sunsari

ARTIFICIAL INTELLIGENCE [ENCT 305]

LAB REPORT: To implement **Breadth First Search (BFS)** and **Depth First Search (DFS)** algorithms for graph traversal and to understand their working in **Artificial Intelligence**

Submitted By:

Abhishek Niraula

Roll no.: PUR080BEI004

Faculty: BEI

Submitted To:

Er. Sujan Karki

Instructor

Purwanchal Campus, Dharan

INTRODUCTION:

Breadth First Search (BFS) is a graph traversal algorithm that starts from a source node and explores the graph level by level. First, it visits all nodes directly adjacent to the source. Then, it moves on to visit the adjacent nodes of those nodes, and this process continues until all reachable nodes are visited.

Depth First Search (DFS) is a graph traversal method that starts from a source vertex and explores each path completely before backtracking and exploring other paths. To avoid revisiting nodes in graphs with cycles, a visited array is used to track visited vertices.

OBJECTIVES:

A) To study about the different searching techniques.
B) To implement Breadth First Search and Depth First Search using Python Programming Language.
C) To compare and study the node traversal techniques used in BFS and DFS algorithms.
D) To learn about representation of undirected graphs in Python and to implement search algorithms.
E) To understand the use of Queue and Stack Data Structures in context to the searching algorithms.

BREADTH FIRST SEARCH

A) ALGORITHM FOR BREADTH FIRST SEARCH

- Step 1: Start by initializing an empty queue **Q**.
- Step 2: Create a list **Visited** to keep track of visited nodes.
- Step 3: Enqueue the starting node **S** into **Q** and mark it as visited.
- Step 4: Repeat steps **5 to 8** until the queue **Q** becomes empty.
- Step 5: Dequeue the front node **N** from **Q**.
- Step 6: Print/Store the node **N** from the traversal list.
- Step 7: For each unvisited neighbor **M** of node **N**:
    - Mark **M** as visited.

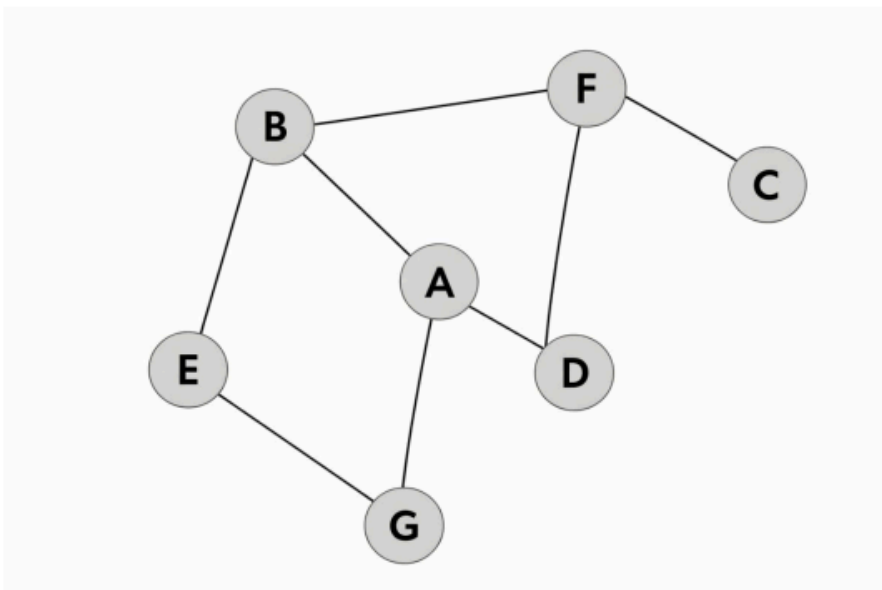
- Enqueue **M** into **Q**.

- Step 8: Continue the loop.
- Step 9: End.

B) PROGRAM CODE FOR BREADTH FIRST SEARCH (BFS)

For the source code, the following graph with the corresponding objective was chosen and the code was written accordingly.

Consider the undirected graph with nodes A, B, C, D, E, F, G and edges as shown in the given figure.



1. Perform Breadth First Search (BFS) on the given graph starting from node A. Show the order in which nodes are visited.

The source code for Breadth First Search is given as follows:

```
#implement Breadth first search using python

graph = {
      # 'A': ['B', 'C'],
      # 'B': ['D', 'E'],
      # 'C': ['F', 'G'],
      # 'D': [],
```

```
        # 'E': [],
        # 'F': [],
        # 'G': [],

        # 'A': ['B', 'C'],
        # 'B': ['D', 'E'],
        # 'C':  ['F'],
        # 'D': ['X'],
        # 'E': ['I'],
        # 'F': ['I', 'H'],
        # 'I': [],
        # 'H': [],
        # 'X': [],

        # LAB REPORT ASSIGNMENT
        'A': ['G', 'D', 'B'],
        'B': ['E', 'F'],
        'G': ['E', 'A'],
        'D': ['A', 'F'],
        'F': ['D', 'C'],
        'E': ['B', 'G'],
        'C': ['F'],
}

queue = []

visited = []


def bfs(visited, queue, node):
        visited.append(node)
        queue.append(node)

        while queue:
                m = queue.pop(0)
                print(m)
                for p in graph[m]:
                        if p not in visited:
                                visited.append(p)
                                queue.append(p)

bfs(visited, queue, 'A')
```

C) OUTPUT OF BREADTH FIRST SEARCH

```
abhiniraula  .../Electronics Engineering/Fifth Semester/Labs  ♥ 21:34  python3 bfs.py
A
G
D
B
E
F
C
```

DEPTH FIRST SEARCH (DFS)

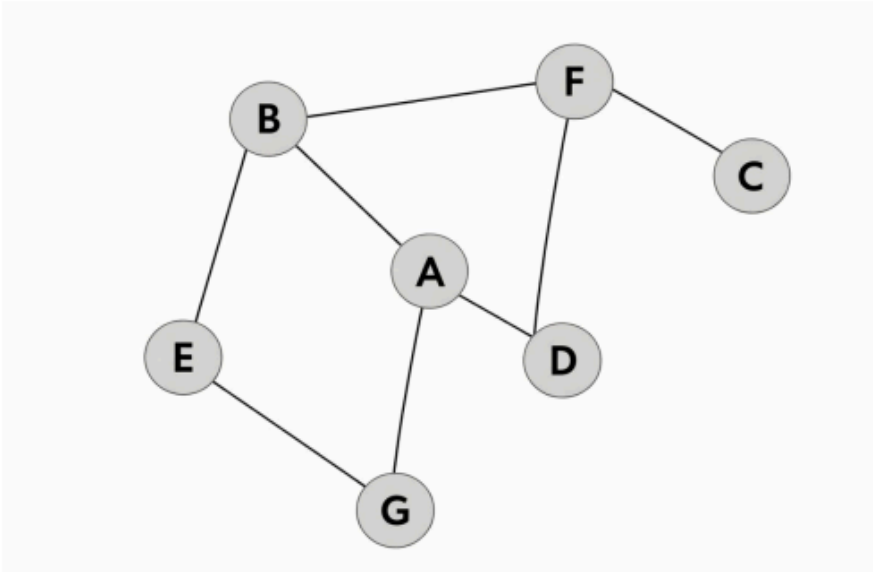
A) ALGORITHM FOR DEPTH FIRST SEARCH

- Step 1: Initialize an empty stack **S**.
- Step 2: Create a list or set **Visited** to keep track of visited nodes.
- Step 3: Push the starting node **s** onto **S** and mark it as visited.
- Step 4: Repeat steps 5 to 8 until **S** becomes empty.
- Step 5: Pop the top node **N** from **S**.
- Step 6: Print/Store the node **N** in the Traversal List.
- Step 7: For each unvisited neighbor **M** of node **N**:
    - Mark **M** as visited.
    - Push **M** onto **S**.
- Step 8: Continue the loop.
- Step 9: End.

B) PROGRAM CODE FOR DEPTH FIRST SEARCH (DFS)

For the source code, the following graph was chosen with the given objectives and the code was written accordingly.

Consider the undirected graph with nodes A, B, C, D, E, F, G and edges as shown in the given figure.



2. Perform Depth First Search (DFS) on the given graph starting from node A. Show the order in which nodes are visited.

The source code for Depth First Search is given as:

```
#implement Depth first search using python

graph = {
    # LAB Exercises
      # 'A': ['B', 'C'],
      # 'B': ['D', 'E'],
      # 'C': ['F', 'G'],
      # 'D': [],
      # 'E': [],
      # 'F': [],
      # 'G': [],
```

```
        # 'A': ['B', 'C'],
        # 'B': ['D', 'E'],
        # 'C':  ['F'],
        # 'D': ['X'],
        # 'E': ['I'],
        # 'F': ['I', 'H'],
        # 'I': [],
        # 'H': [],
        # 'X': [],

        # 'A': ['B', 'C'],
        # 'B': ['D', 'E'],
        # 'C': ['F', 'G'],
        # 'D': ['H', 'I'],
        # 'H': [],
        # 'I': [],
        # 'E': ['J', 'K'],
        # 'J': [],
        # 'K': [],
        # 'F': ['L', 'M'],
        # 'L': [],
        # 'M': [],
        # 'G': ['N', 'O'],
        # 'N': [],
        # 'O': [],

        # LAB REPORT ASSIGNMENT
        'A': ['G', 'D', 'B'],
        'B': ['E', 'F'],
        'G': ['E', 'A'],
        'D': ['A', 'F'],
        'F': ['D', 'C'],
        'E': ['B', 'G'],
        'C': ['F'],
}

stack = []

visited = []


def dfs(visited, stack, graph, node):
```

```
        stack.append(node)

        while stack:
                m = stack.pop()
                if m not in visited:
                        print(m)
                        visited.append(m)
                        for n in reversed(graph[m]):
                                if n not in visited:
                                        stack.append(n)

dfs(visited, stack, graph, 'A')
```

C) OUTPUT OF DEPTH FIRST SEARCH

```
abhiniraula  …/Electronics Engineering/Fifth Semester/Labs  ♥ 21:55  python3 dfs.py
A
G
E
B
F
D
C
```

CONCLUSION

In this lab, we successfully implemented and analyzed two fundamental graph traversal algorithms: Breadth First Search (BFS) and Depth First Search (DFS). We observed how BFS explores nodes level by level using a queue, while DFS follows each path using a stack before backtracking. By applying both algorithms to the given undirected graph, we gained a clear understanding of their traversal order and differences.